Pattern-based Resilience and Responsibility for Graph Queries

Anonymous Author(s)

Abstract

Resilience and responsibility are key concepts in query explainability and query-based causality leading to understand which parts of the property graphs support a given graph query and how robust are query answers under interventions on the data itself. To our knowledge, we are the first to introduce pattern-based resilience and responsibility for graph queries, where the potential interventions on property graphs can range from single edges to arbitrarily complex graph patterns. This unlocks various applications where users can define any graph patterns of interest, and understand how they affect the query. The pattern-based setting raises new technical challenges: intervention matches may overlap with graph query matches and with each other, and an effective intervention may not be contained in any individual query match. To address these challenges, we develop an MILP formulation that connects selected intervention units, their induced edge deletions, and the query matches they destroy. We further derive LP relaxation-based algorithms with approximation guarantees. Experiments on diverse datasets show that our approximation algorithms are effective, efficient, and scalable to billion-edge graphs within reasonable runtime.

ACM Reference Format:
Anonymous Author(s). . Pattern-based Resilience and Responsibility for Graph Queries. In *Proceedings of* . ACM, New York, NY, USA, 14 pages.

1 Introduction

Graphs have become a foundational data model for representing interconnected entities and relationships across a wide range of domains, including finance, security, and social media [53], as well as life sciences [6]. This trend is reflected in the broad ecosystem of graph database systems, including Neo4j [43], Amazon Neptune [4], Oracle PGX [46], SAP HANA Graph [52], RedisGraph [50], Sparksee [34], and Kuzu [31], also in the standardization of graph query languages, including GQL and SQL/PGQ [28, 29]. Among graph data models, *Property Graphs* (PGs) provide a particularly expressive representation: vertices and edges may carry multiple labels and key–value properties, allowing graph topology and semantic attributes to be modeled uniformly [29].

Graph query languages such as openCypher, GQL, and SQL/PGQ provide a declarative interface for querying property graphs through *graph pattern matching*. A graph query specifies a pattern over vertices and edges, together with label constraints and property predicates. Evaluating the query over a property graph returns the matches of the pattern, where each *match* is a subgraph instance of the query pattern on the property graph.

Given the matches returned by a graph query, users are often interested not only in the answer itself, but also in which graph structures support it. Existing work studies *resilience* for regular path queries under atomic edge interventions [3]. Although edges provide a natural intervention granularity, they may be too fine-grained when query answers are large or structurally complex.

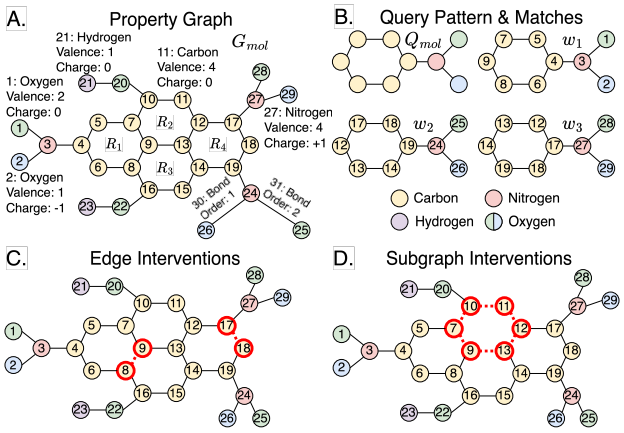


Figure 1: An example of molecular property graph. (A) The graph G_{mol} represents atoms as nodes and bonds as edges; each node has chemical properties for valence and charge; while each edge has chemical properties for bond order. Each carbon ring is denoted as $R_1, \ldots, R_4$. (B) A query pattern Q_{mol} captures a carbon ring attached to a nitrogen group. It induces three matches w_1, w_2, and w_3, each preserving the labels, topology, and related properties of the query pattern. (C) Edge interventions select individual bonds from intervention-pattern matches to invalidate query matches. (D) Subgraph interventions select patterns, e.g., carbon rings, whose edge deletion can destroy all overlapping query matches. Notably, the selected carbon ring R_2 is not included in any query matches.

In many domains, explanations and actions are instead expressed through recurring structures, such as transaction motifs, molecular fragments, or dependency patterns.

Motivated by this, we introduce the *pattern-based resilience* and *pattern-based responsibility*. Given a property graph G, a graph query Q, and an intervention pattern P (graph pattern), an *intervention unit* is a subgraph match of P in G. Pattern-based resilience seeks a minimum set S^* of intervention units whose removal eliminates all matches of Q from G. Given a particular intervention unit u, pattern-based responsibility instead seeks a minimum *contingency set* S^* such that at least one match of Q remains after removing S^*, but no match remains when u is additionally removed. These definitions lift the intervention granularity from individual edges to structured subgraphs that contain multiple edges and overlap with one another. Because selecting a subgraph removes all graph elements covered by its pattern match, pattern-based interventions can be viewed as *bulk interventions*.

Example 1. Consider the molecular property graph shown in Figure 1.A[1]. The graph query induces three query matches, as shown in Figure 1.B. A query match must conform to the topological

[1]For space simplicity, we only show info of nodes 1, 2, 11, 21, 27; and edges 30, 31.

,

structure and associated label constraints specified by the pattern, while also satisfying all predicates over vertex and edge properties. Assume that the intervention pattern specifies a carbon ring in which every carbon atom has valence 4 and charge 0. We then obtain four matches from G_{mol}, denoted by $R_1, \ldots, R_4$. The pattern-based resilience aims to eliminate all the query matches, an optimal solution could be R_2 as highlighted in Figure 1.D. For pattern-based responsibility, if we specify R_2 as the given intervention unit, then the objective is to find a minimum set S^* of intervention units, s.t. after S^* is deleted, the query is still valid, but is invalid if R_2 is also deleted. However, since R_2 by itself can already eliminate all query matches, therefore the optimal solution is $S^* = \emptyset$.

Pattern-based resilience and responsibility support a variety of graph applications and downstream analytical tasks. In financial networks, analysts may ask whether a few laundering motifs, such as circular transfers or fan-in/fan-out patterns, can eliminate all suspicious matches [10]; the identified motifs can support fraud detection and investigation. In molecular graphs, chemists may seek functional groups or fragments whose modification destroys a queried toxic or reactive structure [57], supporting structure–activity analysis and molecular design. In supply-chain and cybersecurity graphs, vulnerable dependency or attack patterns whose removal eliminates critical-risk matches can inform risk assessment and mitigation prioritization [9, 44, 45, 54]. In recommendation graphs, this analysis can determine whether a recommendation has diverse support or depends on a few influential user–item paths or behavioral motifs [55, 58], supporting explanation and robustness auditing. Thus, minimum pattern-based interventions provide a unified means of identifying and quantifying influential subgraphs for detection, explanation, auditing, and intervention prioritization.

• **Challenges**. Pattern-based resilience and responsibility introduce two major challenges. The first challenge is that the optimal solution could be from outside the query matches, thus, we can not find the optimal solution by enumerating through the query matches.

Example 2. Recall the example in Figure 1. The three query matches contain carbon rings such as R_1 and R_4. Under pattern-based intervention, the optimal intervention may be a carbon ring such as the highlighted R_2, which is not contained in any query match but only partially overlaps with them. This shows that optimal interventions must be sought globally over all matches of the intervention pattern instead of query matches.

The second challenge is that the intervention space is implicit and combinatorially large. For pattern-based intervention, the candidates are the *intervention units*, i.e., the matches of intervention pattern P in G, denoted as $\mathcal{M}_P(G)$. Even for a fixed small pattern P, this set can be much larger than the edge set of the graph; under edge-based enumeration, one can bound the number of candidates by $|\mathcal{M}_P(G)| = O(|E(G)|^{|E(P)|})$.

• **Our Contributions**. We introduce pattern-based resilience and responsibility, where we focus on bulk interventions that target intervening in the graph database with subgraphs. Such subgraphs are referred to as *intervention units*, which are specified by intervention patterns, i.e., a graph pattern. Consequently, optimal interventions must be sought globally over all intervention units while accounting for overlaps among them. To tackle these challenges, we develop exact Mixed-Integer Linear Programming (MILP) formulations for pattern-based resilience and responsibility, respectively. The formulations explicitly distinguish the selection of intervention units from the edge deletions they induce. Specifically, unit-selection variables determine which intervention units are deleted, edge-deletion variables represent the union of their effects, and match-level constraints determine which query matches are destroyed. This separation avoids treating overlapping intervention units as independent edge deletions and provides an exact encoding of the pattern-based problems. We note that if the intervention pattern is a single edge, our MILP reduces, up to notational and representational differences, to the existing ILP [33]. In this paper, we focus on the more general setting of structured and potentially overlapping pattern-based interventions.

Although the MILP formulation gives exact characterizations, it can be expensive when the number of query matches and intervention units are large. We therefore develop Linear Programming (LP) relaxation-based approximation algorithms. For pattern-based resilience, we design a randomized rounding strategy based on the LP relaxation of the resilience MILP. A deterministic repair step ensures that the returned intervention set is feasible, yielding an expected $O(\log n)$ approximation, where n is the number of query matches. For pattern-based responsibility, we derive an approximation algorithm by conditioning on surviving query matches w.r.t. the given intervention unit. This follows an observation that the optimum equals the minimum of the conditioned optima over surviving matches. For each candidate, solve and round the corresponding LP relaxation and return the smallest feasible contingency set, which also yields an expected $O(\log n)$ approximation.

Finally, we conduct extensive experiments to evaluate the proposed algorithms on a variety of real-world and synthetic property graphs. The results show that our algorithms return solutions comparable in size to those of baselines, while being up to two orders of magnitude faster, and scale to billion-edge graphs.

In summary, our main contributions are as follows.

(1) We propose novel pattern-based resilience and responsibility, with interventions ranging from single edges to arbitrarily complex graph patterns.
(2) We present exact MILP formulations that use edge-deletion variables to connect intervention-unit selection with query-match destruction, thereby capturing overlaps among them.
(3) We develop LP-relaxation-based randomized-rounding algorithms, with deterministic feasibility repair and expected $O(\log n)$ approximation for fixed query patterns.
(4) We evaluate our methods on diverse real-world and synthetic property graphs. The results show that our algorithms are effective, up to two orders of magnitude faster than baselines, and scalable to billion-edge graphs.

• **Outline**. Section 2 defines the preliminaries to comprehend the rest of the paper. Section 3 defines the novel pattern-based resilience and responsibility. Section 4 introduces the MILP formulation of the proposed problem; and Section 5 elaborates LP relaxation-based approximation algorithms. Section 6 includes extensive evaluation and discusses the results. Finally, Section 7 concludes the paper.

2 Preliminaries

Property Graphs. A *property graph* is a structure $G = (V, E, \eta, \lambda, \nu)$ where: (i) V is a finite set of vertices (nodes) and E is a finite set of edges with $V \cap E = \emptyset$; (ii) $\eta : E \to V \times V$ maps each edge to an ordered pair of vertices; (iii) $\lambda : (V \cup E) \to \mathcal{P}(\mathcal{L})$ assigns a finite set of labels to each vertex or edge; and (iv) $\nu : (V \cup E) \times \mathcal{K} \to \mathcal{N}$ is a partial function assigning property values, i.e., key–value pairs, to vertices and edges [12, 47].

Example 3. In Figure 1.A, G_{mol} is a molecular property graph. Each vertex represents an atom and carries properties such as atom type, valence, and charge. For example, vertex 11 is a carbon atom with valence 4 and charge 0, while edges represent chemical bonds and carry bond order properties. For instance, 3 has order 2.

Graph Query and Graph Pattern. We consider graph queries over property graphs that retrieve subgraphs satisfying a given structural specification and a set of predicates over vertex and edge properties. The structural and predicate constraints of such a query are represented by a graph pattern $Q : \Pi = (V_\Pi, E_\Pi, \phi_V, \phi_E)$, where V_Π and E_Π denote pattern vertices and edges, and ϕ_V and ϕ_E are predicates over vertex and edge labels or properties. A match of R in a property graph G is a subgraph of G that satisfies both the structure induced by (V_Π, E_Π) and the predicates specified by ϕ_V and ϕ_E. Evaluating a graph query amounts to finding all such matches of its associated graph pattern.

Example 4. Figure 1.B shows a query pattern Q_{mol}. The pattern describes a carbon ring attached to a nitrogen atom, where the nitrogen atom is further connected to two oxygen atoms. Properties of different atoms are also specified, e.g., the two oxygen atoms have different valence and charge values.

Matches and Query Answer. A *match* of a graph pattern query Q in a property graph G is an embedding $h = (h_V, h_E)$ that maps query vertices and query edges to vertices and edges in G, while preserving adjacency, labels, and property predicates. That is, for every query edge $(x, y) \in E_Q$, the mapped edge $(h_V(x), h_V(y))$ must exist in G, and all predicates in ϕ_V and ϕ_E must be satisfied. We assume injective matching, i.e., distinct query vertices are mapped to distinct graph vertices. Let $\mathcal{M}_Q(G)$ denote the set of all matches of Q in G. If Q returns full embeddings, then the query answer is $Ans(Q, G) = \mathcal{M}_Q(G)$. If Q projects selected query variables, then $Ans(Q, G)$ is the projection of all matches onto those variables. For a Boolean query, the answer is true iff $\mathcal{M}_Q(G) \neq \emptyset$.

Example 5. In Figure 1.B, Q_{mol} has three matches in G_{mol}, denoted by w_1, w_2, w_3. Match w_1 uses the carbon ring R_1, nitrogen vertex 3, and oxygen vertices 1, 2. Match w_2 uses the carbon ring R_4, nitrogen vertex 24, and oxygen vertices 25, 26. Match w_3 uses the same carbon ring as w_2, but with nitrogen vertex 27 and oxygen vertices 28, 29. Therefore, $\mathcal{M}_{Q_{mol}}(G_{mol}) = \{w_1, w_2, w_3\}$.

3 Pattern-based Resilience and Responsibility

In causality theory, an *intervention* denotes an action that modifies part of a system, and its effect is assessed by evaluating the post-intervention outcome [23, 49]. In relational databases, interventions are typically modeled as tuple deletions [18, 33, 35, 36]. In property graphs, however, interventions naturally arise over graph structures, such as edges [3, 11] and subgraphs. To the best of our knowledge, we are the first to model interventions as pattern-induced subgraph deletions. Each intervention unit is a matched subgraph, represented by its set of edges, whose deletion may change the query outcome. This general formulation supports from single-edge interventions to arbitrarily complex subgraph interventions.

Definition 3.1 (Intervention Pattern and Intervention Unit). An *intervention pattern* P is a graph pattern, which can retrieve subgraph matches from graph database G. The retrieved subgraph matches are referred to as *intervention units*. An *intervention unit* u is a subgraph match of P in G. Therefore, the intervention universe is $\mathcal{M}_P(G)$. For each intervention unit $u \in \mathcal{M}_P(G)$, let $E_u \subseteq E$ denotes the set of edges selected by u. A *candidate intervention set* is any set $S \subseteq \mathcal{M}_P(G)$.

3.1 Resilience for Graph Queries

We now define pattern-based resilience (P-RES) for graph queries. This framework captures both atomic edge interventions and structured subgraph interventions. Atomic edge intervention is obtained when the intervention pattern selects a single edge. More generally, a matched intervention pattern may select several connected edges, such as edges belonging to a path or a sub-graph, so that an intervention unit corresponds to a structured region of the graph rather than to a single edge.

Definition 3.2 (Pattern-based Resilience (P-RES)). Let G be a property graph, Q be a graph query, and P be an intervention pattern. The *Pattern-based resilience* of Q on G with respect to P is

$$\text{P-RES}(Q, G, P) = \min_{S \subseteq \mathcal{M}_P(G)} |S| \quad \text{s.t.} \quad \mathcal{M}_Q(G \setminus S) = \emptyset.$$

A candidate intervention set S satisfying the above condition is called a *valid intervention set*. An optimal valid intervention set is called a *minimum intervention set*.

Example 6. Recall the example shown in Example 1, we discussed the case where the intervention pattern is a single edge, now we further elaborate on the general pattern-based setting. In Figure 1.D, we consider an intervention pattern as a carbon ring, i.e., a 6-cycle consisting of carbon nodes. In G_{mol}, there are four matches: $R_1, \ldots, R_4$. If we choose R_2, then edge $(7, 9)$ and $(12, 13)$ are deleted, which will eliminate all three query matches, thus $S^* = \{R_2\}$ is an optimal solution of the P-RES.

Theorem 3.3 (Hardness of P-RES). *The decision version of P-RES is NP-complete even when Q is a fixed two-edge query and P is a fixed single-edge intervention pattern.*

Proof sketch. Membership in NP is immediate, since Q and P are fixed. For NP-hardness, we reduce from Maximum Directed Cut[20, 48]. Fix Q_2 as a directed path query with two-edges; and fix the single-edge intervention pattern P_1. Thus each intervention deletes exactly one matched edge. Given a directed graph $D = (V, A)$ and an integer B, construct $G := D$, $Q := Q_2$, $P := P_1$, and $k := |A| - B$, If D has a directed cut (S, T) with at least B arcs from S to T, delete all arcs not crossing from S to T. At most $|A| - B = k$ arcs are deleted, and all remaining arcs go from S to T. Hence no remaining arc can be followed by another remaining

arc, so no match of Q_2 remains. Conversely, suppose at most k single-edge interventions delete a set $F \subseteq A$ such that no match of Q_2 remains. Let $A' := A \setminus F$. Then $|A'| \geq |A| - k = B$. Since $G[A']$ has no directed path of length two, no vertex is both the head of a remaining arc and the tail of a remaining arc. Define $S := \{v \in V : \exists w \in V,\ (v, w) \in A'\}$, and $T := V \setminus S$. For every $(u, v) \in A'$, we have $u \in S$. If $v \in S$, then there exists $(v, w) \in A'$, yielding a forbidden two-edge path $u \to v \to w$. Therefore $v \in T$, and every arc in A' crosses from S to T. Thus (S, T) is a directed cut of size at least $|A'| \geq B$. Therefore the constructed P-RES instance is a yes-instance iff the Maximum Directed Cut instance is a yes-instance. Hence P-RES is NP-hard. Together with membership in NP, it is NP-complete. □

3.2 Responsibility for Intervention Units

We next define pattern-based responsibility (P-RSP), which measures how strongly a fixed intervention unit contributes to the query answer. The idea is to ask whether there exists a contingency set such that the query still holds after applying it, but becomes false once the fixed intervention unit is additionally applied. The smaller the required contingency set, the larger the causal responsibility of the fixed intervention unit.

Definition 3.4 (Pattern-based Responsibility (P-RSP)). Let G be a property graph, Q be a graph query, P be an intervention pattern, and let $u^\circ \in \mathcal{M}_P(G)$ be a fixed intervention unit. A set $S \subseteq \mathcal{M}_P(G) \setminus \{u^\circ\}$ is called a *contingency set* for u° if

$$\mathcal{M}_Q(G \setminus S) \neq \emptyset \quad \text{and} \quad \mathcal{M}_Q\big(G \setminus (S \cup \{u^\circ\})\big) = \emptyset.$$

The *pattern-based responsibility* of u° for Q on G with respect to P is

$$\text{P-RSP}(Q, G, P, u^\circ) = \begin{cases} \dfrac{1}{1 + \min |S|}, & \text{if } S \text{ exists,} \\ 0, & \text{otherwise,} \end{cases}$$

where S ranges over contingency sets for u°. We call u° an *actual cause* if there exists a contingency set. In particular, u° is a *counterfactual cause* if $S = \emptyset$ [35].

Example 7. Again consider the edge-intervention pattern P_{edge} from Figure 1.C. Let u° be edge $(8, 9)$, deleting it alone will only eliminate w_1, thus it needs a contingency set to be an actual cause. Such a set could be $\{(17, 18)\}$, where $(17, 18)$ eliminates w_2 and w_3. Since it is minimum, an optimal contingency set for $u^\circ = (8, 9)$ is $S^* = \{(17, 18)\}$.

Theorem 3.5 (Hardness of P-RSP). *The decision version of P-RSP is NP-complete with fixed intervention unit u°.*

Proof sketch. Membership in NP follows by verifying, for a guessed set $S \subseteq \mathcal{M}_P(G) \setminus \{u^\circ\}$, whether $\mathcal{M}_Q(G \setminus S) \neq \emptyset$ and $\mathcal{M}_Q(G \setminus (S \cup \{u^\circ\})) = \emptyset$. For NP-hardness, we reduce from P-RES, which is NP-hard by Theorem 3.3. Given an instance (G, Q, P, k) of P-RES, construct G' by adding one fresh query match that is deleted only by a fresh intervention unit u°, while u° does not affect any edge of the original graph G. Then a set S of size at most k is a valid P-RES solution for G iff it is a contingency set for u° in G': after deleting S, the only remaining query match is the fresh one, and applying u° removes it; conversely, any contingency set for u° must already delete all original query matches, since u° only deletes the fresh match. Thus P-RES $\leq_p$ P-RSP, and P-RSP is NP-hard. Therefore, the decision problem is NP-complete. □

4 MILP Formulation for P-RES & P-RSP

We present Mixed Integer Linear Program (MILP) formulations for pattern-based query resilience (P-RES) and responsibility (P-RSP). To address the challenges raised in the pattern-based setting, we introduce an edge variable to track the overlapping relationship between intervention units and query matches, and between intervention units themselves. Let $U = \mathcal{M}_P(G)$ be the set of intervention units, with $m = |U|$ and let $W = \mathcal{M}_Q(G)$ be the set of query matches, with $n = |W|$. For every intervention unit $u \in U$, let $E_u \subseteq E$ be the edge set of u. For every query match $w \in W$, let $E_w \subseteq E$ be the edge set of w.

4.1 MILP for P-RES

The MILP for P-RES is:

$$\min \quad \sum_{u \in U} x_u \tag{1}$$

$$\text{s.t.} \quad d_e \leq \sum_{\substack{u \in U \\ e \in E_u}} x_u \qquad \forall e \in E, \tag{2}$$

$$d_e \geq x_u \qquad \forall u \in U,\ \forall e \in E_u, \tag{3}$$

$$\sum_{e \in E_w} d_e \geq 1 \qquad \forall w \in W, \tag{4}$$

$$x_u \in \{0, 1\} \qquad \forall u \in U, \tag{5}$$

$$0 \leq d_e \leq 1 \qquad \forall e \in E. \tag{6}$$

Variables. The P-RES MILP requires two variables to keep track of the intervention units and ensure side effects are counted. For each intervention unit $u \in U$, introduce a binary variable **(i)** $x_u \in \{0, 1\}$, where $x_u = 1$ means that u is selected. For each edge $e \in E$, introduce a fractional variable **(ii)** $0 \leq d_e \leq 1$. d_e can still ensure exact solutions as fractional variables: assuming we initialize all d_e to be 0, by constraint (3), when $x_u = 0$, all d_e will satisfy the constraint; when $x_u = 1$, constraint (3) becomes $d_e \geq 1$, resulting in $d_e = 1$. Therefore, d_e is used to track the selection of edges.
Constraints. Constraint (2) ensures that if an edge is selected, it should be included in at least one intervention unit. Constraint (3) ensures that if an intervention unit is selected, then all its edges should be selected. Constraints (2) and (3) link selected intervention units to deleted edges. Constraint (4) enforces suppression of query matches. For every query match $w \in W$, at least one edge in E_w must be deleted. Since a query match is destroyed once any of its required edges is deleted, these constraints ensure that all query matches are eliminated.
Objective. The objective of P-RES MILP is to minimize the number of intervention units to delete.

Theorem 4.1 (Correctness of P-RES MILP). *The optimal value of the P-RES MILP is equal to the minimum number of intervention units whose deletion destroys all query matches in W.*

Proof sketch. Let $S = \{u \in U \mid x_u = 1\}$ be the intervention set selected by any feasible ILP solution. Constraints (2) and (3)

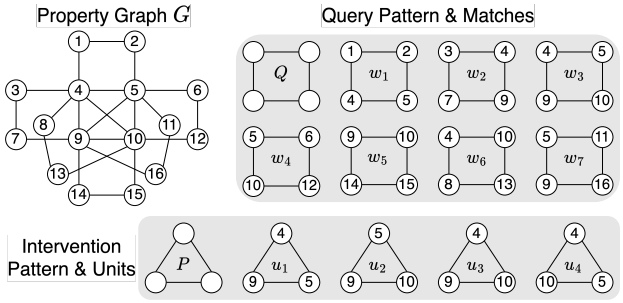


Figure 2: A property graph G (we only show the topology of G for visualization simplicity.) For a 4-cycle graph query Q, seven query matches are returned as $w_1, \ldots, w_7$. The intervention pattern P is a 3-cycle (triangle) pattern with four intervention matches $u_1, \ldots, u_4$.

enforce $d_e = 1$ if there exists a $u \in S$, s.t. $e \in E_u$. Thus, the variables d_e exactly represent the physical edges deleted by the selected intervention units. Constraint (4) then requires that every query match $w \in W$ contains at least one deleted edge, and hence every query match is destroyed. Therefore, every feasible P-RES MILP solution gives a valid intervention set with the same objective value. Conversely, given any valid intervention set $S \subseteq U$, set $x_u = 1$ iff $u \in S$, and $d_e = 1$ iff $e \in \bigcup_{u \in S} E_u$. These assignments satisfy constraints (2) and (3), and since S destroys every query match, they also satisfy constraint (4). The objective value is exactly $|S|$. Hence the P-RES MILP optimum is equal to the optimal P-RES value. □

Example 8. Consider the example shown in Figure 2. To eliminate all seven matches $w_1, \ldots, w_7$ of the 4-cycle query Q, assume we select set $S = \{u_1, u_2, u_3\}$, then by constraint (2) and (3), edges $(4, 9)$, $(4, 5)$, $(5, 9)$, $(5, 10)$, $(9, 10)$, $(4, 10)$ are selected and all their d_e are set to 1. Constraint (4) is then satisfied since all seven matches have at least one $d_e = 1$. Since any subset of S will not satisfy constraint (4), the optimum of P-RES MILP is three, explicitly $\{u_1, u_2, u_3\}$.

Discussion. Our MILP is designed for pattern-based interventions, where each unit may contain multiple edges and different units may overlap by sharing edges. When the intervention pattern is restricted to a single edge, each unit-selection variable can be identified with its corresponding edge-deletion variable, allowing the intermediate linking layer to be eliminated. In this special case, our P-RES MILP reduces, up to notational differences, to the established resilience ILP [33]. This correspondence shows that our formulation is consistent with the single-edge setting. We therefore focus our theoretical and experimental evaluation on structured and potentially overlapping pattern-based interventions.

4.2 MILP for P-RSP

Instead of asking for a minimum set to eliminate all query matches as P-RES, the P-RSP asks for a minimum set to enable a given intervention unit to be an *actual cause*, i.e., all the query matches will be eliminated iff the given unit is deleted.

Given an intervention unit u°, let $b_e = 1$ if $e \in E_{u^\circ}$, and $b_e = 0$ otherwise. Then b_e records whether edge e would be deleted by the fixed candidate cause u°. The MILP for u°'s responsibility is:

$$\min \quad \sum_{u \in U \setminus \{u^\circ\}} x_u \tag{7}$$

$$\text{s.t.} \quad d_e \le \sum_{\substack{u \in U \setminus \{u^\circ\} \\ e \in E_u}} x_u \qquad \forall e \in E, \tag{8}$$

$$d_e \ge x_u \qquad \forall u \in U \setminus \{u^\circ\},\ \forall e \in E_u, \tag{9}$$

$$\sum_{e \in E_w} (d_e + b_e) \ge 1 \qquad \forall w \in W, \tag{10}$$

$$\sum_{w \in W} z_w = 1, \tag{11}$$

$$z_w \le \sum_{e \in E_w} b_e \qquad \forall w \in W, \tag{12}$$

$$z_w + d_e \le 1 \qquad \forall w \in W,\ \forall e \in E_w, \tag{13}$$

$$x_u \in \{0, 1\} \qquad \forall u \in U \setminus \{u^\circ\}, \tag{14}$$

$$0 \le d_e \le 1 \qquad \forall e \in E, \tag{15}$$

$$0 \le z_w \le 1 \qquad \forall w \in W. \tag{16}$$

Variables. For each intervention unit $u \in U \setminus \{u^\circ\}$, introduce a binary variable **(i)** $x_u \in \{0, 1\}$, where $x_u = 1$ means that u is selected into the contingency set. For each edge $e \in E \setminus E_{u^\circ}$, introduce a fractional variable **(ii)** $0 \le d_e \le 1$, where d_e are forced to be $\{0, 1\}$ with the same reason as in P-RES MILP (§ 4.1). It tracks whether e is deleted by the contingency set. For each query match $w \in W$, introduce a fractional variable **(iii)** $0 \le z_w \le 1$, z_w is used to track the query matches that *cannot* be deleted by the contingency set, but can be deleted after deleting u°. Briefly speaking, we can choose any query match w with $z_w > 0$ as the one surviving query match. A full explanation is in the supplementary materials [2].

Constraints. Constraints (8) and (9) serve the same purposes as in P-RES MILP (§ 4.1), one difference is that they only consider intervention units excluding u°. Constraint (10) requires that after applying both the contingency set and u°, every query match is destroyed. That is, every query match w must lose at least one edge either because it is deleted by the contingency set or because it is deleted by u°. Constraint (11) selects one query match as the surviving match before u° is deleted. Constraint (12) ensures that the selected query match can be deleted by u°. Finally, constraint (13) ensures that no edge of the selected query match is deleted by the contingency set.

Objective. The objective of P-RSP MILP is to minimize the size of the contingency set, i.e., the number of intervention units to delete.

Theorem 4.2 (Correctness of P-RSP MILP). *For a fixed intervention unit $u^\circ \in U$, the P-RSP MILP computes the minimum size of a contingency set for u°.*

Proof sketch. As in the P-RES MILP, constraints (8) and (9) ensure that $d_e = 1$ exactly for the edges deleted by S. Constraint (10) requires every query match to lose at least one edge after applying both S and u°. Therefore, all query matches are destroyed after deleting the candidate cause. It remains to ensure that u° is necessary. Constraint (11) selects one query match w with $z_w > 0$ as a surviving match before u° is deleted. Constraint (13) ensures that no edge of this selected match is deleted by S, so the query still

Algorithm 1 LP-Rounding for P-RES (LpRdRes)

Input: Intervention units U, query matches W, size of fixed query $q = |Q|$, edge sets $\{E_u\}_{u\in U}$, $\{E_w\}_{w\in W}$.
Output: Intervention set $S \subseteq U$.
1: Build $\mathcal{L} \leftarrow \textsc{Lookup}(W, U)$.
Step 1: LP relaxation
2: Solve the LP relaxation of the P-RES MILP and obtain (x^*, d^*).
Step 2: Randomized rounding
3: Initialize $S \leftarrow \emptyset$ and $\lambda \leftarrow \max\left\{1,\ q \ln\left(\frac{n}{LP^*_{\text{P-RES}}}\right)\right\}$.
4: **for** each $u \in U$ **do**
5: Set $p_u \leftarrow \min\{1, 1 - (1 - x^*_u)^\lambda\}$.
6: Add u to S independently with probability p_u.
7: Set $D \leftarrow \bigcup_{u\in S} E_u$.
8: **for** each $w \in W$ **do**
9: **if** $E_w \cap D = \emptyset$ **then**
10: $u_w \leftarrow \textsc{SelectUnit}(w, \mathcal{L})$.
11: Add u_w to S.
12: Update $D \leftarrow D \cup E_{u_w}$.
13: **return** S.

holds after deleting the contingency set. Constraint (12) ensures that the selected match is overlapped with u° with at least one edge, and hence is destroyed once u° is deleted. Therefore, every feasible P-RSP MILP solution corresponds to a valid contingency set of the same size. Conversely, given any valid contingency set S, set $x_u = 1$ iff $u \in S$. $d_e = 1$ iff e is deleted by S, and choose as $z_w > 0$ a query match that survives S but is destroyed after deleting u°. These assignments satisfy all constraints and have objective value $|S|$. Hence the P-RSP MILP optimum is exactly the minimum contingency-set size. □

Example 9. Recall the example shown in Figure 2, assume u_3 is the given intervention unit, i.e., u°, then $b_e^{4,9}, b_e^{4,10}, b_e^{9,10}$ are 1. Assume we select a contingency set as $\{u_1, u_2\}$, by constraints (8) and (9), $d_e^{4,9}, d_e^{4,5}, d_e^{5,9}, d_e^{5,10}, d_e^{9,10}$ are forced to be 1. Thus, constraint (10) is satisfied by above d_e and b_e. Assume we set the z_w^6 and z_w^7 of w_6 and w_7 both to $\frac{1}{2}$, both constraints (11) and (13) are satisfied, but z_w^7 violates constraint (12) due to the overlap between w_7 and u_2, resulting in $z_w^7 + d_e^{5,9} = \frac{1}{2} + 1 > 1$; meanwhile, w_7 is not overlapped with $u_\circ$. Finally, w_6 will be the surviving query match that is deleted by $u^\circ = u_3$, and the contingency set $\{u_1, u_2\}$ is valid and minimum.

5 LP Relaxation and Approximation

The MILP formulations above provide exact solutions for P-RES and P-RSP. However, solving these MILPs exactly can be very expensive when the number of query matches or intervention-pattern matches is large. To make the computation scalable, we next develop Linear Programming (LP) relaxation-based approximation algorithms. We relax the binary variable x_u for both P-RES and P-RSP to fractional variables in $[0, 1]$. Based on the relaxations, we derive efficient randomized rounding with logarithmic approximation guarantees.

5.1 LP Relaxation for P-RES MILP

LP relaxation. For P-RES, we relax the binary variables in constraint (5) to $0 \le x_u \le 1, \forall u \in U$. All other constraints and the objective remain unchanged. Let (x^*, d^*) be an optimal LP solution and let $LP^*_{\text{P-RES}} = \sum_{u\in U} x^*_u$ be the LP optimum. Since the LP relaxation enlarges the feasible region of the MILP, its optimum is a lower bound on the integral optimum: $LP^*_{\text{P-RES}} \le \text{OPT}_{\text{P-RES}}$.

Rounding Algorithm. Algorithm 1 gives our LP-rounding approximation algorithm (LpRdRes) for P-RES. Its design consists of two main steps. Step 1 solves an LP relaxation to obtain fractional guidance for selecting intervention units. Step 2 applies randomized rounding to obtain an integral solution with feasibility repair. The algorithm first builds a sparse lookup structure $\mathcal{L}$ over the materialized query matches and intervention units (line 1). It avoids storing all pairs (u, w) such that $E_u \cap E_w \neq \emptyset$. Details of Lookup are given in the supplementary materials [2]. In Step 1, the algorithm solves the LP relaxation of the P-RES MILP and obtains the optimal fractional solution (x^*, d^*) (line 2). The values x^*_u quantify the fractional preference for selecting each intervention unit u and guide the subsequent integral solution construction. In Step 2, the algorithm first initializes the output set S and sets the rounding parameter $\lambda = \max\left\{1, q \ln\left(\frac{n}{LP^*_{\text{P-RES}}}\right)\right\}$ (line 3), where $n = |W|$ is the number of query matches. For each intervention unit $u \in U$, it then computes the sampling probability $p_u = \min\{1, 1 - (1 - x^*_u)^\lambda\}$ (line 5) and independently adds u to S with probability p_u (line 6). This probability transformation amplifies larger LP values while keeping the probability in $[0, 1]$. Thus, units that are more important in the LP solution are more likely to be selected during rounding. The second part of Step 2 repairs the rounded set to ensure feasibility. The algorithm computes the deleted-edge set $D = \bigcup_{u\in S} E_u$ (line 7) to check whether any match remains after rounding (line 8- 9). For every such leftover match, the algorithm invokes $\textsc{SelectUnit}(w, \mathcal{L})$ to find one intervention unit u_w whose edge set intersects E_w (line 10). Details of SelectUnit are in the supplementary materials [2]. The selected unit u_w is then added to the intervention set S (line 11), and the deleted-edge set is updated accordingly (line 12). Finally, the algorithm returns a feasible intervention set S (line 13).

Theorem 5.1. *For every feasible P-RES instance, LpRdRes returns a feasible intervention set. Under the fixed-query assumption, the expected size of the returned set is an $O(\log n)$-approximation.*

Proof sketch. For a query match $w \in W$, let $N(w) = \{u \in U \mid E_u \cap E_w \neq \emptyset\}$ be the intervention units that can destroy w. From the LP constraints, every query match must receive one unit of fractional edge deletion: $\sum_{e\in E_w} d^*_e \ge 1$. This implies

$$1 \le \sum_{e\in E_w} d^*_e \le \sum_{u\in N(w)} |E_u \cap E_w| x^*_u \le \alpha \sum_{u\in N(w)} x^*_u,$$

where $\alpha = |E_w|$. Hence, $\sum_{u\in N(w)} x^*_u \ge \frac{1}{q}$. In the rounding step, each unit u is selected with probability $p_u = 1 - (1 - x^*_u)^\lambda$, where we set $\lambda = \max\left\{1,\ q \ln\left(\frac{n}{LP^*_{\text{P-RES}}}\right)\right\}$. Thus, the probability that w survives the rounding step is

$$\begin{aligned}\Pr[w \text{ survives}] &= \prod_{u\in N(w)} (1 - p_u) = \prod_{u\in N(w)} (1 - x^*_u)^\lambda \\ &\le \exp\left(-\lambda \sum_{u\in N(w)} x^*_u\right) \le \frac{LP^*_{\text{P-RES}}}{n}.\end{aligned}$$

Therefore, the expected number of query matches left after rounding is at most $LP^*_{\text{P-RES}}$. The repair step adds at most one intervention unit for each leftover query match, so the expected number of intervention units for repair is at most $LP^*_{\text{P-RES}}$. Meanwhile, since $p_u = 1 - (1 - x^*_u)^\lambda \le \lambda x^*_u$, the expected number of rounded units

is at most $\sum_{u\in U} p_u \le \lambda \sum_{u\in U} x^*_u = \lambda LP^*_{\text{P-RES}}$. Hence the expected output size is at most $(\lambda+1)LP^*_{\text{P-RES}} \le (\lambda+1)\text{OPT}_{\text{P-RES}}$. Since Q is fixed, $q = O(1)$, and therefore $\lambda = O(\log n)$. Thus the expected approximation ratio is $O(\log n)$. Finally, feasibility follows from the repair step: every query match not destroyed by rounding is explicitly repaired by adding one intervention unit that shares an edge with it. Hence all query matches are eliminated. □

Example 10. Recall the example shown in Figure 2, assume the LP output of all intervention units are $u_1 = 0.63$, $u_2 = 0.21$, $u_3 = 0.98$, $u_4 = 0.12$. Then the LP optimum $LP^*_{\text{P-RES}} = 2.05$. Meanwhile, $q = 4$ and $n = 7$, which gives us $\lambda = 4.2$. then the probabilities of selecting each intervention unit are 0.98, 0.71, 1.0, and 0.48 for $p^1_u \dots p^4_u$, respectively. Assume the rounding step selects u_1 and u_3, then w_7 is still surviving, which will invoke the repair procedure to select one unit that eliminates w_7, e.g., u_2. Since the final intervention set $\{u_1, u_2, u_3\}$ is already optimal, the returned S is bounded by $O(\log n)$-approximation.

5.2 LP-Rounding Approximation for P-RSP

From the MILP to survivor-conditioned LPs. Recall the P-RSP MILP from Section 4.2, where we select one query match as the *surviving* match, s.t. it can *not* be deleted by the contingency set, but can be deleted by the given intervention unit $u_\circ$. The selection of the surviving query match is managed through variable z_w.

An intuition of an approximation algorithm for P-RSP is to relax the P-RSP MILP. However, a direct LP relaxation of the P-RSP MILP would relax $x_u \in \{0,1\}$ to $0 \le x_u \le 1$ while retaining the fractional variables d_e and z_w. This relaxation is not directly suitable for rounding: the constraint $\sum_{w\in W} z_w = 1$ allows the surviving match's mass to be distributed fractionally among multiple matches, without guaranteeing that any single match is completely preserved.

Therefore, we instead exploit the disjunctive structure encoded by the surviving match variables in the P-RSP MILP. Given the candidate cause u°, define

$$W^+(u^\circ) = \{r \in W \mid E_r \cap E_{u^\circ} \neq \emptyset\}.$$

These are the query matches that can be destroyed by u° and are therefore possible surviving matches before u° is deleted.

In an integral solution of the P-RSP MILP, the binary variables x_u force every d_e to be integral. Consequently, if $z_r > 0$, constraint (13) gives $d_e = 0$, $\forall e \in E_r$. Thus, r genuinely survives the contingency set. Moreover, constraint (12) ensures that $r \in W^+(u^\circ)$. We therefore fix one such match r as the survivor by setting $z_r = 1$ and $z_w = 0$ for every $w \neq r$. It follows that the P-RSP MILP admits the exact decomposition

$$\text{OPT}_{\text{P-RSP}} = \min_{r\in W^+(u^\circ)} \text{OPT}_{\text{P-RSP}}(r),$$

where $\text{OPT}_{\text{P-RSP}}(r)$ is the minimum contingency-set size under the condition that r survives.

Conditioning on a surviving match. Fix a candidate surviving match $r \in W^+(u^\circ)$. Setting $z_r = 1$ in constraint (13) gives $d_e = 0$, $\forall e \in E_r$. Together with constraint (9), this implies $x_u = 0$ for every intervention unit whose edge set overlaps with E_r. We therefore define the intervention units that preserve r as

$$U_r = \{u \in U \setminus \{u^\circ\} \mid E_u \cap E_r = \emptyset\}.$$

We also define

$$W^0(u^\circ) = \{w \in W \mid E_w \cap E_{u^\circ} = \emptyset\}.$$

Every match in $W^0(u^\circ)$ is unaffected by u° and must therefore be destroyed by the contingency set. For every match $w \notin W^0(u^\circ)$, we have $\sum_{e\in E_w} b_e \ge 1$, so constraint (10) is automatically satisfied after u° is deleted.

Hence, after fixing r, the original P-RSP MILP reduces to the following r-conditioned formulation:

$$\begin{aligned}
\min \quad & \sum_{u\in U_r} x_u && && (17)\\
\text{s.t.} \quad & d_e \le \sum_{\substack{u\in U_r\\ e\in E_u}} x_u && \forall e \in E, && (18)\\
& d_e \ge x_u && \forall u \in U_r,\ \forall e \in E_u, && (19)\\
& \sum_{e\in E_w} d_e \ge 1 && \forall w \in W^0(u^\circ), && (20)\\
& x_u \in \{0,1\} && \forall u \in U_r, && (21)\\
& 0 \le d_e \le 1 && \forall e \in E. && (22)
\end{aligned}$$

This formulation resembles the P-RES MILP, but has two restrictions that are specific to responsibility. First, only the matches in $W^0(u^\circ)$ must be destroyed by the contingency set. Second, only intervention units in U_r may be selected, ensuring that the designated surviving match r is preserved.

LP relaxation. For each $r \in W^+(u^\circ)$, we relax constraint (21) to $0 \le x_u \le 1$, $\forall u \in U_r$. All other constraints and the objective remain unchanged. Let $(x^{r,*}, d^{r,*})$ be an optimal solution and let $LP^*_{\text{P-RSP}}(r) = \sum_{u\in U_r} x^{r,*}_u$ be the optimum of the r-conditioned LP. The overall LP lower bound is $LP^*_{\text{P-RSP}} = \min_{r\in W^+(u^\circ)} LP^*_{\text{P-RSP}}(r)$, where infeasible conditioned LPs are ignored.

Lemma 5.2. *For every feasible P-RSP instance, $LP^*_{\text{P-RSP}} \le OPT_{\text{P-RSP}}$.*

Proof sketch. Let S^* be an optimal contingency set. Since S^* makes u° a counterfactual cause, there exists a match r^* that survives after deleting S^* but is destroyed after additionally deleting u°. Therefore, $E_{r^*} \cap E_{u^\circ} \neq \emptyset$, and hence $r^* \in W^+(u^\circ)$. Since r^* survives S^*, no unit in S^* intersects r^*. Thus, $S^* \subseteq U_{r^*}$. Moreover, every match in $W^0(u^\circ)$ is unaffected by u° and must already be destroyed by S^*. Therefore, S^* induces a feasible integral solution to the r^*-conditioned formulation with objective value $|S^*|$. It follows that $LP^*_{\text{P-RSP}} \le LP^*_{\text{P-RSP}}(r^*) \le |S^*| = \text{OPT}_{\text{P-RSP}}$. □

Rounding Algorithm. Algorithm 2 (LpRdRsp) gives our LP rounding approximation algorithm for P-RSP. The algorithm first builds a sparse lookup structure $\mathcal{L}$ over the query matches and intervention units (line 1). The lookup structure supports edge-overlap checks and repair-unit selection without materializing the complete intervention-unit–match incidence matrix. The algorithm partitions the query matches into W^+, containing the matches destroyed by u°, and W^0, containing the matches unaffected by u° (lines 2–3). If W^+ is empty, no match can certify that u° is necessary, so the instance is infeasible (lines 4– 5). If W^0 is empty, deleting u° alone destroys every query match, and the empty contingency set is optimal (lines 6– 7). Next, the algorithm enumerates every $r \in W^+$

Algorithm 2 LP-Rounding for P-RSP (LpRdRsp)

Input: Intervention units U, candidate cause u°, query matches W, fixed query size $q = |Q|$, edge sets $\{E_u\}_{u\in U}$ and $\{E_w\}_{w\in W}$.
Output: Contingency set $S \subseteq U \setminus \{u^\circ\}$, or Infeasible.

1: Build $\mathcal{L} \leftarrow \textsc{Lookup}(W, U)$.
2: $W^+ \leftarrow \{r \in W \mid E_r \cap E_{u^\circ} \neq \emptyset\}$.
3: $W^0 \leftarrow \{w \in W \mid E_w \cap E_{u^\circ} = \emptyset\}$.
4: **if** $W^+ = \emptyset$ **then**
5: **return** Infeasible.
6: **if** $W^0 = \emptyset$ **then**
7: **return** $\emptyset$.
8: $S^* \leftarrow \perp$.
9: **for** each $r \in W^+$ **do**
10: $U_r \leftarrow \{u \in U \setminus \{u^\circ\} \mid E_u \cap E_r = \emptyset\}$.
Step 1: LP Relaxation
11: Solve the r-conditioned LP and obtain $(x^{r,*}, d^{r,*})$.
12: **if** the LP is infeasible **then**
13: **continue**.
14: $\lambda_r \leftarrow \max\left\{1,\ q\ln\left(\frac{|W^0|}{LP^*_{\text{P-RSP}}(r)}\right)\right\}$. $S_r \leftarrow \emptyset$.
Step 2: Randomized rounding
15: **for** each $u \in U_r$ **do**
16: $p_u \leftarrow \min\{1, 1-(1-x_u^{r,*})^{\lambda_r}\}$.
17: Add u to S_r independently with probability p_u.
18: $D_r \leftarrow \bigcup_{u\in S_r} E_u$.
19: **for** each $w \in W^0$ **do**
20: **if** $E_w \cap D_r = \emptyset$ **then**
21: $u_w \leftarrow \textsc{SelectUnit}(w, \mathcal{L}, U_r)$.
22: $S_r \leftarrow S_r \cup \{u_w\}$.
23: $D_r \leftarrow D_r \cup E_{u_w}$.
24: **if** $S^* = \perp$ or $|S_r| < |S^*|$ **then**
25: $S^* \leftarrow S_r$.
26: **if** $S^* = \perp$ **then**
27: **return** Infeasible.
28: **return** S^*.

as the match that survives the contingency set (line 9). For each candidate r, LpRdRsp consists of two main steps: it first constructs U_r, containing only intervention units that preserve r (line 10), and solves the corresponding r-conditioned LP (line 11). An infeasible LP means that r cannot serve as the surviving match and is skipped. Secondly, for every feasible conditioned LP, LpRdRsp independently selects each $u \in U_r$ with probability $p_u = \min\{1, 1-(1-x_u^{r,*})^{\lambda_r}\}$, where $\lambda_r = \max\left\{1,\ q\ln\left(\frac{|W^0|}{LP^*_{\text{P-RSP}}(r)}\right)\right\}$. Since every rounded unit belongs to U_r, randomized rounding cannot destroy the candidate surviving match r. After rounding, the algorithm computes the deleted-edge set D_r (line 18). For every match in W^0 that remains undestroyed, it selects one repair unit from U_r that intersects the match (lines 19– 22). Such a unit exists whenever the conditioned LP is feasible. Finally, LpRdRsp returns the smallest contingency set obtained over all candidate surviving matches (lines 24– 28).

Theorem 5.3. *For every feasible P-RSP instance, LpRdRsp returns a valid contingency set. Under the fixed-query assumption, the expected size of the returned set is an $O(\log n)$-approximation.*

Proof sketch. Let S^* be an optimal contingency set, and let r^* be a query match that survives S^* but is destroyed after additionally deleting u°. Consider the iteration of LpRdRsp corresponding to r^*, and let $L = LP^*_{\text{P-RSP}}(r^*)$. By Lemma 5.2, $L \le \text{OPT}_{\text{P-RSP}}$. For a match $w \in W^0(u^\circ)$, let $N_{r^*}(w) = \{u \in U_{r^*} \mid E_u \cap E_w \neq \emptyset\}$ be the survivor-compatible intervention units that can destroy w. From the conditioned LP constraints, $\sum_{e\in E_w} d_e^{r^*,*} \ge 1$. This implies

$$1 \le \sum_{e\in E_w} d_e^{r^*,*} \le \sum_{u\in N_{r^*}(w)} |E_u \cap E_w| x_u^{r^*,*} \le q \sum_{u\in N_{r^*}(w)} x_u^{r^*,*}.$$

Hence, $\sum_{u\in N_{r^*}(w)} x_u^{r^*,*} \ge \frac{1}{q}$. In the rounding step, each unit $u \in U_{r^*}$ is selected with probability $p_u = 1-(1-x_u^{r^*,*})^{\lambda_{r^*}}$, where $\lambda_{r^*} = \max\left\{1,\ q\ln\left(\frac{|W^0(u^\circ)|}{L}\right)\right\}$. Thus, the probability that w survives the rounding step is

$$\begin{aligned}\Pr[w \text{ survives}] &= \prod_{u\in N_{r^*}(w)} (1-p_u) = \prod_{u\in N_{r^*}(w)} (1-x_u^{r^*,*})^{\lambda_{r^*}} \\ &\le \exp\left(-\lambda_{r^*} \sum_{u\in N_{r^*}(w)} x_u^{r^*,*}\right) \le \frac{L}{|W^0(u^\circ)|}.\end{aligned}$$

Therefore, the expected number of matches left after rounding is at most L. The repair step adds at most one intervention unit for each leftover match, so the expected number of repair units is also at most L. Meanwhile, since $p_u = 1-(1-x_u^{r^*,*})^{\lambda_{r^*}} \le \lambda_{r^*} x_u^{r^*,*}$, the expected number of rounded units is at most $\sum_{u\in U_{r^*}} p_u \le \lambda_{r^*} \sum_{u\in U_{r^*}} x_u^{r^*,*} = \lambda_{r^*} L$. Hence, the expected size of the solution produced in the r^* iteration is at most $(\lambda_{r^*}+1)L \le (\lambda_{r^*}+1)\text{OPT}_{\text{P-RSP}}$. Since LpRdRsp returns the smallest solution over all candidate survivors, its expected output size is no larger. Moreover, $L \ge 1/q$ whenever $W^0(u^\circ) \neq \emptyset$. Since Q is fixed, $q = O(1)$, and $|W^0(u^\circ)| \le n$, we have $\lambda_{r^*} = O(\log n)$. Thus, the expected approximation ratio is $O(\log n)$. Finally, all selected units belong to U_{r^*}, so r^* survives the contingency set. The repair step destroys every match in $W^0(u^\circ)$, while every remaining match overlaps with u° and is destroyed after u° is additionally deleted. Therefore, the returned set is a valid contingency set for u°. □

Example 11. Recall the example in Figure 2, and let $u^\circ = u_1$. Then $W^+ = \{w_1, w_2, w_3, w_7\}$ contains the candidate surviving matches, while $W^0 = \{w_4, w_5, w_6\}$ must be destroyed by the contingency set. Consider the iteration that fixes $r = w_1$ as the survivor. The units preserving w_1 are $U_r = \{u_2, u_3\}$, and the conditioned LP gives $x_{u_2}^{r,*} = x_{u_3}^{r,*} = 1$, with $LP^*_{\text{P-RSP}}(r) = 2$. Since $q = 4$ and $|W^0| = 3$, we obtain $\lambda_r \approx 1.62$, and both units are selected with probability 1. The resulting contingency set $S_r = \{u_2, u_3\}$ destroys w_4, w_5, w_6 while preserving w_1. After additionally deleting u_1, all remaining matches, including w_1, are destroyed. The algorithm similarly examines w_2, w_3, and w_7 as survivors and returns the smallest feasible set, which has size 2.

6 Experimental Study

In this section, we evaluate the effectiveness, efficiency, and scalability of our approaches by answering three questions: **(Q1)** How small are the intervention sets returned for P-RES and the contingency sets returned for P-RSP? **(Q2)** How efficiently do the algorithms solve P-RES and P-RSP? **(Q3)** How well do the algorithms scale with increasing input size? We compare our LP-rounding algorithms against the greedy and random baselines introduced in Section 6.1.
Environment. We implement all algorithms in Python 3.11.9. LP and MILP models are constructed using PuLP [37] and solved using the COIN-OR Branch-and-Cut (CBC) solver [17]. Query matches

Table 1: Statistics of datasets

dataset	# nodes	# edges	ave_deg	# node labels	# edge labels
StarWars	260	611	4.7	6	4
wwc2019	2,486	14,799	11.9	5	9
StackOverflow	6,193	11,540	3.7	5	6
Fincen	7,524	40,835	10.9	3	5
ILPG	411,787	1,117,528	5.4	5	10
ICIJ	2,016,523	3,339,267	3.3	5	14
LDBC	282,386,021	1,775,513,185	6.3	11	20

and intervention units are retrieved from Neo4j 5.26.0. All experiments are conducted on a machine equipped with a 24-core AMD EPYC 9224 processor, 512 GB of RAM, and 1 TB of disk storage. **Codes are available in [1]**.

6.1 Experimental Setup

Datasets. Table 1 summarizes the datasets. We evaluate effectiveness and efficiency on six real-world property graphs. The StarWars dataset describes characters, species, planets, and their relationships in the Star Wars universe [30, 40]. The wwc2019 dataset records teams, players, matches, and tournament results from the 2019 FIFA Women's World Cup [38]. The StackOverflow dataset captures users, questions, answers, comments, and tags from the Stack Overflow programming community [41]. The Fincen dataset represents suspicious financial transactions and connections among financial institutions disclosed by the FinCEN Files investigation [16, 39]. The Italian Legislative Property Graph (ILPG) models Italian legislation, including the structure of legal documents and their citation, modification, and abrogation relationships [14, 15]. The ICIJ dataset describes offshore entities, officers, intermediaries, addresses, and their relationships disclosed through ICIJ investigations [27, 42]. For scalability, we use synthetic social-network property graphs generated at different scale factors by the LDBC Social Network Benchmark datagen [5, 32].

Algorithms. The exact MILPs establish the optimization framework for P-RES and P-RSP, but solving them has exponential worst-case complexity and becomes prohibitively expensive on large instances. Their LP relaxations therefore serve as the foundation of our scalable approximation algorithms, and we report only approximation results. As discussed in Section 4.1, under single-edge interventions, our MILPs reduce to the established formulations for resilience and responsibility. Rather than repeating evaluations in this special case, we focus on the more general pattern-based setting with structured and potentially overlapping intervention units. We compare our LP-rounding algorithms, LpRdRes and LpRdRsp, against two families of baselines. The first consists of LP-guided baselines, including LpGdRes and LpGdRsp for P-RES and P-RSP, respectively. These methods solve the same LP relaxation as our algorithms but replace the proposed rounding procedure with greedy or random selection. The second consists of naive baselines, including GdRes, GdRsp, RmRes, and RmRsp, which apply greedy or random selection directly to the intervention units without guidance from the LP solution. These baselines allow us to separately evaluate the benefits of LP guidance and the proposed rounding strategy.

Evaluation Metrics. We evaluate the algorithms using three metrics. **(i) Resilience** is the minimum number of intervention units whose deletion invalidates all query matches. Accordingly, we report the size of the intervention set returned by each P-RES algorithm, where smaller is better. **(ii) Responsibility** of a designated intervention unit is determined by the size of its minimum contingency set, as defined in Definition 3.4. Since the responsibility score is inversely correlated with the contingency size, our plots report the returned contingency-set size instead of the score; a smaller value therefore indicates greater responsibility. **(iii) Runtime** measures the execution time of each algorithm and is used to evaluate efficiency and scalability. For randomized algorithms, we repeat each experiment five times and report the average results.

Query and Intervention Patterns. Both query and intervention patterns specify topological and label constraints together with predicates over vertex and edge properties; a subgraph match is retained only when all these constraints are satisfied. Different pattern selections produce different numbers of query matches and intervention units. We therefore evaluate the algorithms across configurations of increasing size. The query patterns are selected to produce match counts at powers of ten, while the total number of intervention units is bounded by one tenth of the corresponding number of query matches. The complete pattern configurations are available in our code repository [1].

6.2 Q1. Effectiveness

We evaluate the solution quality of the algorithms for both P-RES and P-RSP. For P-RES, we report the *resilience*, measured by the size of the returned intervention set. For P-RSP, we report the returned contingency-set size, which determines the *responsibility* according to Definition 3.4. Smaller values indicate better solutions for both problems. We further examine the contributions of the LP relaxation and the proposed rounding strategy separately by comparing against the naive and LP-guided baselines, respectively.

Effect of LP guidance. Figures 3 and 4 compare our LP-rounding algorithms with the naive greedy and random baselines (RmRes and RmRsp). For P-RES, LpRdRes consistently produces intervention sets comparable to, and generally smaller than, those of GdRes. At the largest setting of each dataset, its returned sets are 1.3%–4.0% smaller than those of GdRes. For P-RSP, LpRdRsp and GdRsp also return contingency sets of nearly identical size, with differences below 3% at the largest settings. This is notable because greedy selection explicitly updates the marginal coverage after every selected unit, whereas the LP captures the global interaction among intervention units, deleted edges, and query matches in a single optimization step.

The advantage over naive random selection is substantially larger and increases with the number of query matches. At the largest settings, RmRes returns intervention sets between 4.5× and 36.4× larger than those of LpRdRes, while the contingency sets returned by RmRsp are between 5.5× and 37.2× larger than those of LpRdRsp. Naive random selection ignores both the fractional importance of individual units and their overlaps with previously selected units. It consequently selects more redundant units and leaves more query matches to the repair step. In contrast, the LP solution assigns larger values to units that contribute more effectively to satisfying the global coverage constraints, providing substantially better guidance as the candidate space grows.

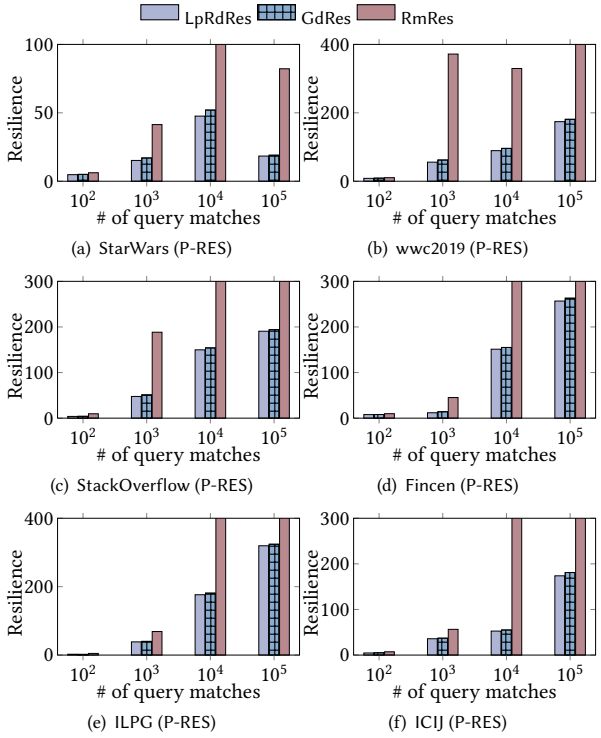


Figure 3: Effectiveness of the LP relaxation for P-RES.

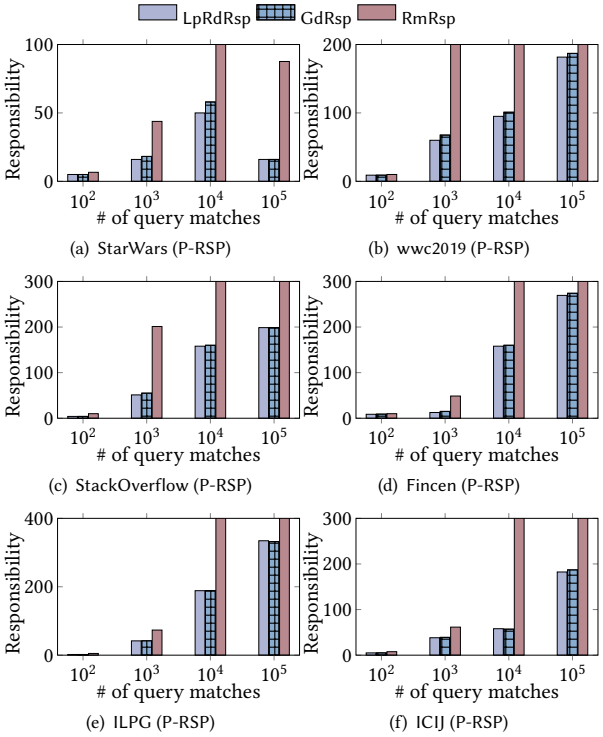


Figure 4: Effectiveness of the LP relaxation for P-RSP.

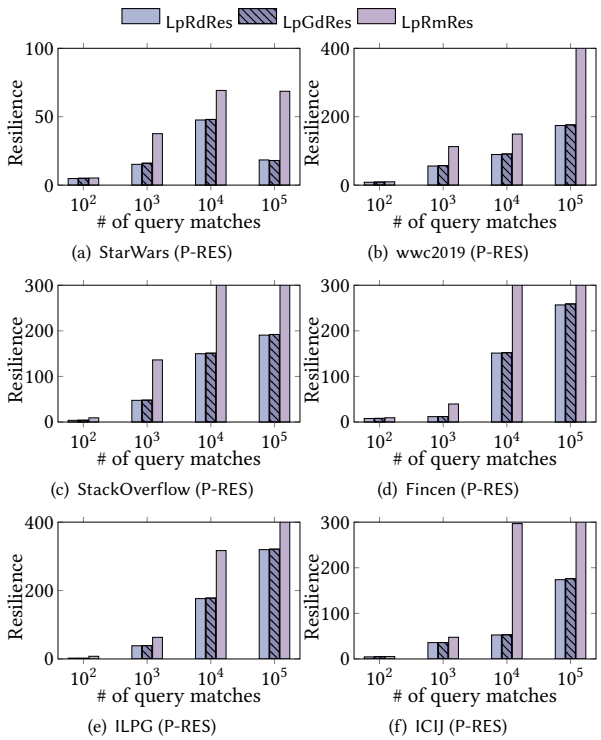


Figure 5: Effectiveness of the rounding strategy for P-RES.

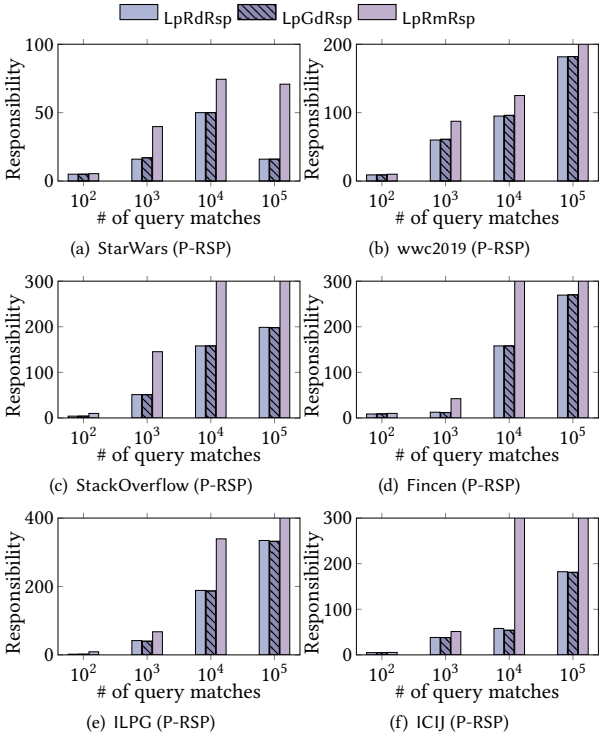


Figure 6: Effectiveness of the rounding strategy for P-RSP.

Effect of the rounding strategy. Figures 5 and 6 compare our rounding strategy with greedy and random selection (LpRmRes and LpRmRsp) over the same LP-guided solution space. The solution sizes of LpRdRes and LpGdRes differ by less than 1% in aggregate across all configurations. The same behavior is observed for P-RSP, where LpRdRsp and LpGdRsp also differ by less than 1% in

aggregate. Thus, the proposed rounding preserves almost the same solution quality as iterative LP-greedy selection.

By contrast, LP-random selection deteriorates rapidly on larger instances. At the largest settings, LpRmRes returns sets between 3.7× and 14.9× larger than those of LpRdRes, and LpRmRsp returns contingency sets between 4.4× and 15.0× larger than those of LpRdRsp. Although these methods use the same LP relaxation, random selection does not adequately preserve the relative importance encoded by the fractional variables. Our probability $p_u = 1 - (1 - x_u)^{\lambda}$ instead amplifies the selection of high-valued units while retaining sufficient coverage across different query matches. This reduces redundant selections and limits the number of units introduced during repair. Overall, the results consistently show that LP guidance provides the main improvement over naive random selection, while the proposed rounding strategy achieves solution quality comparable to greedy selection for both P-RES and P-RSP.

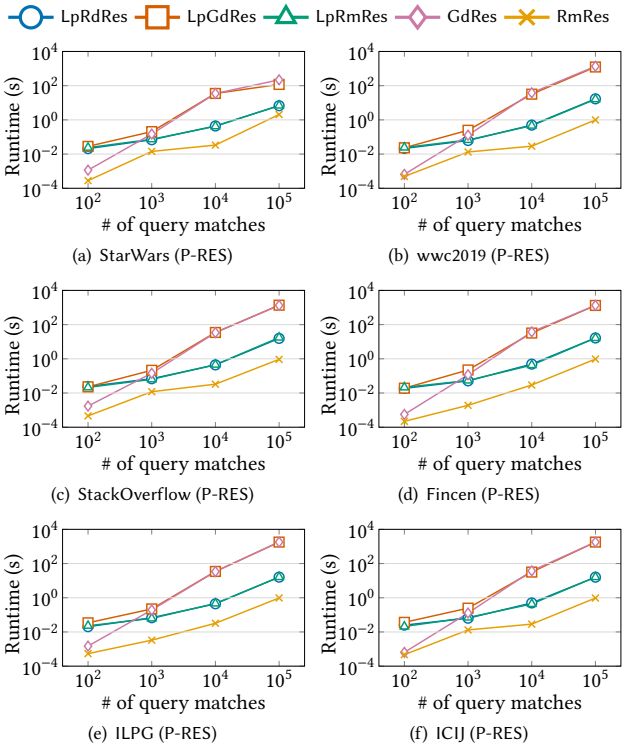


Figure 7: Efficiency of P-RES.

6.3 Q2. Efficiency

Figures 7 and 8 report the runtime of all algorithms as the number of query matches increases. For both P-RES and P-RSP, the differences are small on the smallest instances, where all methods terminate quickly. As the input size increases, however, the LP-rounding and random methods remain substantially faster than greedy methods.

At the largest setting of each dataset, LpRdRes is between 17.6× and at least 112× faster than LpGdRes, and between 32.6× and at least 112× faster than GdRes. The same trend holds for P-RSP. In particular, both greedy variants reach the 1,800-second timeout on ILPG and ICIJ, whereas LpRdRes and LpRdRsp finish in approximately 16 seconds. The high cost of greedy selection results from its iterative mechanism: after selecting an intervention unit,

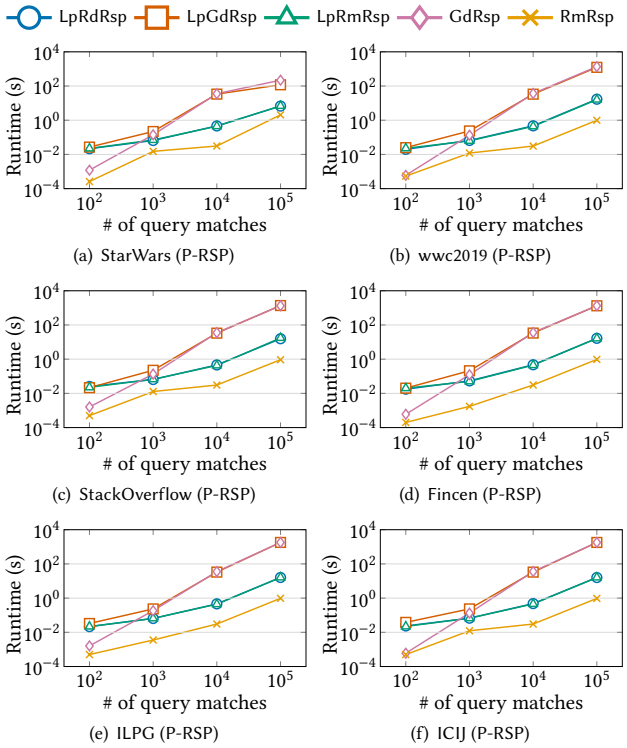


Figure 8: Efficiency of P-RSP.

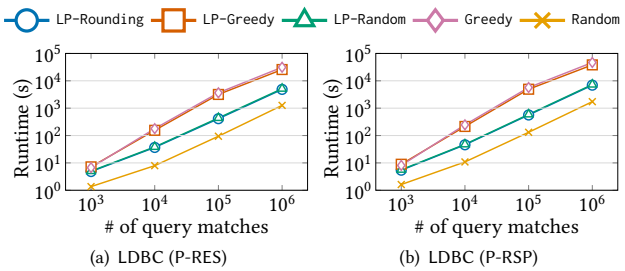
it must update the remaining query matches and recompute or maintain the marginal gains of the other candidates. As the number of matches and units grows, these repeated updates dominate the runtime. LP-guided greedy additionally incurs the cost of solving the LP before this iterative selection.

In contrast, our algorithms solve the LP and then round all fractional variables in a single pass. Only the query matches left uncovered by rounding are processed during repair. This explains why LpRdRes and LpRmRes have nearly overlapping runtime curves, with less than 1% difference on average; the same observation holds for LpRdRsp and LpRmRsp. These pairs solve the same LP, while the cost difference between their rounding procedures is negligible relative to LP construction and optimization. As shown by the effectiveness results, however, our rounding produces much smaller solutions than LP-random selection at essentially the same runtime.

RmRes and RmRsp are the fastest because they avoid both LP optimization and iterative marginal-gain computation. At the largest settings, they are approximately 3.3×–17.1× faster than our algorithms. This speed comes at a substantial effectiveness cost: their returned sets are up to 36.4× larger for P-RES and 37.2× larger for P-RSP. Overall, LP rounding provides the best balance between efficiency and effectiveness: it achieves solution quality comparable to greedy selection, runs orders of magnitude faster than the greedy methods, and substantially improves solution quality over random methods with similar runtime.

6.4 Q3. Scalability

We evaluate scalability on the synthetic LDBC graphs by increasing the number of query matches from 10^3 to 10^6. Figure 9 reports the runtime for P-RES and P-RSP on these configurations.



(a) LDBC (P-RES) (b) LDBC (P-RSP)

Figure 9: Scalability of LP-rounding and baselines.

The runtime of our LP-rounding algorithms grows approximately with the input size. When the number of query matches increases by 1,000×, the runtime of LpRdRes increases from 4.83 to 4,856.32 seconds, corresponding to a 1,005× increase. Similarly, LpRdRsp increases from 5.41 to 6,928.51 seconds, or by 1,281×. Thus, both algorithms process configurations containing one million query matches, with P-RSP incurring moderately higher cost due to its additional survivor-conditioned computation.

The greedy methods scale considerably less favorably. From 10^3 to 10^6 query matches, the runtimes of LP-greedy and greedy increase by approximately 3,609× and 4,657×, respectively, for P-RES, and by 4,279× and 5,769× for P-RSP. At one million matches, LpRdRes is 5.3× faster than LpGdRes and 6.4× faster than GdRes. Likewise, LpRdRsp is 5.5× faster than LpGdRsp and 6.7× faster than GdRsp. This increasing gap is caused by the repeated marginal-gain updates of greedy selection, whose cost grows with both the number of candidates and the number of remaining query matches. Our rounding instead performs a single sampling pass after LP optimization and repairs only the uncovered matches.

LP-random remains close to our algorithms in runtime because it solves the same LP and differs only in its inexpensive selection step. At one million matches, its runtime is within 6% of LP-rounding for both problems, although the effectiveness experiments show that it returns much larger solutions. Random is the fastest, being approximately 3.8× and 4.0× faster than our algorithms for P-RES and P-RSP, respectively, at the largest setting, but it provides substantially poorer solution quality. Overall, LP rounding exhibits substantially better empirical scalability than greedy selection while retaining the effectiveness benefits of LP-guided optimization.

7 Related Work

Necessary Explanations. The notion of necessity and explanation was introduced by Halpern et al. [13, 23, 24] in the context of causal reasoning, where an explanation is defined in terms of counterfactual causality and minimal interventions on the input. This line of work, rooted in causal AI rather than data management [22], formalizes how causes and *responsibilities* can be attributed to outcomes through structural causal models. While our work is inspired by this notion of intervention, we operate in the context of graph databases, where our bulk interventions take the form of subgraph deletions on the underlying graph instance rather than modifications to a causal model. Finding explanations for and understanding the causes of query results is a problem that has also driven data management research [7, 8, 21]. Many approaches to explanation for relational databases have been explored. Perhaps the most interesting for our problem is modifying the input [25, 26, 51, 56]. Still, unlike these approaches, which operate on individual relational tuples, our bulk interventions consist of pattern-induced subgraphs, i.e., intervention units.

Resilience and Causal Responsibility. The concept of causal responsibility was further adapted into relational databases [18, 33, 35, 36] to formalize the notion of query *resilience*. For a Boolean query, *resilience* measures how robust the query outcome is under deletions of input tuples. While resilience quantifies the minimum amount of change needed to flip the query outcome, *responsibility* in relational databases ranks how strongly an individual tuple contributes to the outcome [33, 35, 36]. Our work adapts the concepts of *resilience* and *responsibility* to graph databases. Due to the structural characteristics of graph instances, we extend the notion of intervention unit to a pattern-induced subgraph, which in turn helps discover important structures inside the property graphs. A dichotomy for the complexity of both resilience and responsibility on self-join free queries under set semantics was established in [18]. New results followed on queries with self-joins [19], characterizing the complexity landscape of resilience for conjunctive queries with self-joins and highlighting the structural reasons behind the hardness introduced by self-joins. This is particularly relevant in the context of graph queries, where self-joins arise naturally even in simple graph patterns. Recently, the complexity of resilience for RPQ (regular path queries) in graph databases has been investigated and found tractable for local languages [3, 11]. In our paper, we focus on efficient algorithms to compute both resilience and responsibility for non-recursive graph queries.

ILP for Resilience and Responsibility. A unified approach to compute the resilience and the causal responsibility in relational databases was established in [33] using ILP formulations of the problems, as well as an (MI)LP relaxation. While we also formulate our problems in MILP (which we then relax for approximation purposes), our formulations take into consideration the challenges brought by pattern-based bulk interventions. Since an intervention unit only destroys a query match if they share at least one edge, we introduce new variables and constraints taking into account the concept of subgraph intervention units, and linking these intervention units to the edges that constitute them. Further, our MILP formulations for the optimal P-RES and P-RSP can be reduced to the respective ILPs in the existing work [33] in the special case where the intervention unit is a single edge, since in such a setting, the edges forming the intervention unit are equal to the intervention unit itself.

8 Conclusion

We propose novel pattern-based resilience (P-RES) and pattern-based responsibility (P-RSP), which consider bulk interventions with subgraph matches as the intervention unit. We develop exact MILP formulations to tackle the intervention overlap and the combinatorial solution space challenges. Moreover, we propose LP rounding algorithms with expected $O(\log n)$ approximation for both P-RES and P-RSP. Extensive experiments across diverse datasets demonstrate that our approximation algorithms scale to billion-edge property graphs while achieving a favorable balance between intervention-set size and runtime efficiency.

References

[1] 2026. Code and datasets. *"https://anonymous.4open.science/r/C4GQ-0507/"*.

[2] 2026. Full version. *"https://anonymous.4open.science/r/PGQC-full-CB04/"*.

[3] Antoine Amarilli, Wolfgang Gatterbauer, Neha Makhija, and Mikaël Monet. 2025. Resilience for Regular Path Queries: Towards a Complexity Classification. *Proceedings of the ACM on Management of Data* 3, 2 (2025), 1–18.

[4] Amazon Web Services. [n. d.]. Amazon Neptune. https://aws.amazon.com/neptune/ Accessed: 2026-07-14.

[5] Renzo Angles, János Benjamin Antal, Alex Averbuch, Peter Boncz, Orri Erling, Andrey Gubichev, Vlad Haprian, Moritz Kaufmann, Josep Lluis Larriba-Pey, Norbert Martínez-Bazan, József Marton, Marcus Paradies, Minh-Duc Pham, Arnau Prat-Pérez, Mirko Spasic, Benjamin A. Steer, Gábor Szárnyas, and Jack Waudby. 2020. The LDBC Social Network Benchmark. doi:10.48550/arXiv.2001.02299 arXiv:2001.02299.

[6] Albert-László Barabási and Zoltán N. Oltvai. 2004. Network Biology: Understanding the Cell's Functional Organization. *Nature Reviews Genetics* 5, 2 (2004), 101–113. doi:10.1038/nrg1272

[7] Leopoldo Bertossi. 2021. Specifying and computing causes for query answers in databases via database repairs and repair-programs. *Knowl. Inf. Syst.* 63, 1 (Jan. 2021), 199–231. doi:10.1007/s10115-020-01516-6

[8] Leopoldo Bertossi and Babak Salimi. 2017. From Causes for Database Queries to Repairs and Model-Based Diagnosis and Back. *Theor. Comp. Sys.* 61, 1 (July 2017), 191–232. doi:10.1007/s00224-016-9718-9

[9] Jennifer Blackhurst, Kevin S. Dunn, and Craig W. Craighead. 2018. Supply chain vulnerability assessment: A network based visualization and clustering analysis approach. *Journal of Purchasing and Supply Management* 24, 1 (2018), 21–30.

[10] Jovan Blanuša, Maximo Cravero Baraja, Andreea Anghel, Luc von Niederhäusern, Erik Altman, Haris Pozidis, and Kubilay Atasu. 2024. Graph Feature Preprocessor: Real-time Subgraph-based Feature Extraction for Financial Crime Detection. In *Proceedings of the 5th ACM International Conference on AI in Finance*. doi:10.1145/3677052.3698674

[11] Manuel Bodirsky, Žaneta Semanišinová, and Carsten Lutz. 2024. The Complexity of Resilience Problems via Valued Constraint Satisfaction Problems. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '24)*. Association for Computing Machinery, New York, NY, USA, Article 14, 14 pages.

[12] Angela Bonifati, George H. L. Fletcher, Hannes Voigt, and Nikolay Yakovets. 2018. *Querying Graphs*. Synthesis Lectures on Data Management, Vol. 10. Morgan & Claypool Publishers, USA.

[13] Hana Chockler and Joseph Y. Halpern. 2004. Responsibility and Blame: A Structural-Model Approach. *Journal of Artificial Intelligence Research* 22 (2004), 93–115. doi:10.1613/jair.1391

[14] Andrea Colombo, Anna Bernasconi, and Stefano Ceri. 2024. *Italian Legislative Property Graph*. doi:10.5281/zenodo.11210265

[15] Andrea Colombo, Anna Bernasconi, and Stefano Ceri. 2024. Modelling Legislative Systems into Property Graphs to Enable Advanced Pattern Detection. doi:10.48550/arXiv.2406.14935 arXiv:2406.14935.

[16] Emilia Díaz-Struck and Agustin Armendariz. 2020. Download FinCEN Files Transaction Data. https://www.icij.org/investigations/fincen-files/download-fincen-files-transaction-data/.

[17] John J. H. Forrest and Robin D. Lougee-Heimer. 2005. CBC User Guide. In *Emerging Theory, Methods, and Applications*. INFORMS, 257–277. doi:10.1287/educ.1053.0020

[18] Cibele Freire, Wolfgang Gatterbauer, Neil Immerman, and Alexandra Meliou. 2015. The Complexity of Resilience and Responsibility for Self-Join-Free Conjunctive Queries. *Proceedings of the VLDB Endowment* 9, 3 (2015), 156–167.

[19] Cibele Freire, Wolfgang Gatterbauer, Neil Immerman, and Alexandra Meliou. 2020. New Results for the Complexity of Resilience for Binary Conjunctive Queries with Self-Joins. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS'20)* (Portland, OR, USA). Association for Computing Machinery, New York, NY, USA, 271–284. doi:10.1145/3375395.3387647

[20] Michael R. Garey, David S. Johnson, and Larry Stockmeyer. 1976. Some Simplified NP-Complete Graph Problems. *Theoretical Computer Science* 1, 3 (1976), 237–267.

[21] Boris Glavic, Alexandra Meliou, and Sudeepa Roy. 2021. Trends in Explanations: Understanding and Debugging Data-driven Systems. *Found. Trends Databases* 11, 3 (Aug. 2021), 226–318. doi:10.1561/1900000074

[22] Joseph Y. Halpern. 2015. Cause, responsibility and blame: a structural-model approach. *Law, Probability and Risk* 14, 2 (06 2015), 91–118. arXiv:https://academic.oup.com/lpr/article-pdf/14/2/91/5080997/mgu020.pdf doi:10.1093/lpr/mgu020

[23] Joseph Y. Halpern and Judea Pearl. 2005. Causes and Explanations: A Structural-Model Approach. Part I: Causes. *The British Journal for the Philosophy of Science* 56, 4 (2005), 843–887.

[24] Joseph Y. Halpern and Judea Pearl. 2005. Causes and Explanations: A Structural-Model Approach. Part II: Explanations. *British Journal for the Philosophy of Science* 56 (2005), 889–911. doi:10.1093/bjps/axi147

[25] Melanie Herschel, Mauricio A. Hernández, and Wang-Chiew Tan. 2009. Artemis: a system for analyzing missing answers. *Proc. VLDB Endow.* 2, 2 (Aug. 2009), 1550–1553. doi:10.14778/1687553.1687588

[26] Jiansheng Huang, Ting Chen, AnHai Doan, and Jeffrey F. Naughton. 2008. On the provenance of non-answers to queries over extracted data. *Proc. VLDB Endow.* 1, 1 (Aug. 2008), 736–747. doi:10.14778/1453856.1453936

[27] International Consortium of Investigative Journalists. 2026. ICIJ Offshore Leaks Database. https://offshoreleaks.icij.org/pages/database. Accessed: 2026-07-13.

[28] ISO/IEC. 2023. *ISO/IEC 9075-16:2023 Information Technology—Database Languages SQL—Part 16: Property Graph Queries (SQL/PGQ)*. Technical Report ISO/IEC 9075-16:2023. International Organization for Standardization. https://www.iso.org/standard/79473.html

[29] ISO/IEC. 2024. *ISO/IEC 39075:2024 Information Technology—Database Languages—GQL*. Technical Report ISO/IEC 39075:2024. International Organization for Standardization. https://www.iso.org/standard/76120.html

[30] Kaggle. [n. d.]. The Star Wars Dataverse. https://www.kaggle.com/datasets/jsphyg/star-wars.

[31] KuzuDB Contributors. [n. d.]. Kuzu: An Embedded Property Graph Database. https://github.com/kuzudb/kuzu Project archived on October 10, 2025; accessed: 2026-07-14.

[32] Linked Data Benchmark Council. 2025. LDBC Social Network Benchmark Data Generator. https://github.com/ldbc/ldbc_snb_datagen_hadoop.

[33] Neha Makhija and Wolfgang Gatterbauer. 2023. A Unified Approach for Resilience and Causal Responsibility with Integer Linear Programming (ILP) and LP Relaxations. *Proceedings of the ACM on Management of Data* 1, 4, Article 228 (2023), 27 pages.

[34] Norbert Martínez-Bazán, Víctor Muntés-Mulero, Sergio Gómez-Villamor, Jordi Nin, Mario Sánchez-Martínez, and Josep-Lluís Larriba-Pey. 2007. DEX: High-Performance Exploration on Large Graphs for Information Retrieval. In *Proceedings of the 16th ACM Conference on Information and Knowledge Management*. 573–582. doi:10.1145/1321440.1321521

[35] Alexandra Meliou, Wolfgang Gatterbauer, Joseph Y. Halpern, Christoph Koch, Katherine F. Moore, and Dan Suciu. 2010. Causality in Databases. *IEEE Data Engineering Bulletin* 33, 3 (2010), 59–67.

[36] Alexandra Meliou, Wolfgang Gatterbauer, Katherine F. Moore, and Dan Suciu. 2010. The Complexity of Causality and Responsibility for Query Answers and Non-Answers. *Proceedings of the VLDB Endowment* 4, 1 (2010), 34–45.

[37] Stuart Mitchell, Michael OSullivan, and Iain Dunning. 2011. Pulp: a linear programming toolkit for python. *The University of Auckland, Auckland, New Zealand* 65, 25 (2011), 2011.

[38] Neo4j Graph Examples. 2019. Women's World Cup 2019 Graph Example. https://github.com/neo4j-graph-examples/wwc2019.

[39] Neo4j Graph Examples. 2020. ICIJ FinCEN Files Graph Example. https://github.com/neo4j-graph-examples/fincen.

[40] Neo4j Graph Examples. 2022. Star Wars Graph Example. https://github.com/neo4j-graph-examples/star-wars.

[41] Neo4j Graph Examples. 2024. Stack Overflow Graph Example. https://github.com/neo4j-graph-examples/stackoverflow.

[42] Neo4j Graph Examples. 2025. ICIJ Offshore Leaks Graph Example. https://github.com/neo4j-graph-examples/icij-offshoreleaks. Accessed: 2026-07-13.

[43] Neo4j, Inc. [n. d.]. Neo4j Graph Database. https://neo4j.com/product/neo4j-graph-database/ Accessed: 2026-07-14.

[44] Steven Noel and Sushil Jajodia. 2008. Optimal IDS Sensor Placement and Alert Prioritization Using Attack Graphs. *Journal of Network and Systems Management* 16, 3 (2008), 259–275.

[45] Ariella Olswang, Rami Puzis, and Yuval Elovici. 2022. Prioritizing vulnerability patches in large networks. *Expert Systems with Applications* 188 (2022), 115992.

[46] Oracle Corporation. [n. d.]. Oracle Integrated Graph Database. https://www.oracle.com/database/integrated-graph-database/ Accessed: 2026-07-14.

[47] Amedeo Pachera, Angela Bonifati, and Andrea Mauri. 2025. User-Centric Property Graph Repairs. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.

[48] Christos H. Papadimitriou and Mihalis Yannakakis. 1991. Optimization, Approximation, and Complexity Classes. *J. Comput. System Sci.* 43, 3 (1991), 425–440.

[49] Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, USA.

[50] RedisGraph Contributors. [n. d.]. RedisGraph: A Graph Database Module for Redis. https://github.com/RedisGraph/RedisGraph Project no longer maintained; accessed: 2026-07-14.

[51] Sudeepa Roy and Dan Suciu. 2014. A formal approach to finding explanations for database queries. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (Snowbird, Utah, USA) *(SIGMOD '14)*. Association for Computing Machinery, New York, NY, USA, 1579–1590. doi:10.1145/2588555.2588578

[52] Michael Rudolf, Marcus Paradies, Christof Bornhövd, and Wolfgang Lehner. 2013. The Graph Story of the SAP HANA Database. In *Proceedings of the 15th Conference on Database Systems for Business, Technology, and Web (BTW) (Lecture Notes in Informatics, Vol. P-214)*. 403–420.

[53] Siddhartha Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M. Tamer Özsu. 2020. The Ubiquity of Large Graphs and Surprising Challenges of Graph Processing: Extended Survey. *The VLDB Journal* 29 (2020), 595–618. doi:10.1007/s00778-019-00548-x

[54] Stephan M. Wagner and Nikrouz Neshat. 2010. Assessing the vulnerability of supply chains using graph theory. *International Journal of Production Economics* 126, 1 (2010), 121–129.

[55] Xiang Wang, Dingxian Wang, Canran Xu, Xiangnan He, Yixin Cao, and Tat-Seng Chua. 2019. Explainable Reasoning over Knowledge Graphs for Recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[56] Eugene Wu and Samuel Madden. 2013. Scorpion: explaining away outliers in aggregate queries. *Proc. VLDB Endow.* 6, 8 (June 2013), 553–564. doi:10.14778/2536354.2536356

[57] Zhenxing Wu, Jike Wang, Hongyan Du, Dejun Jiang, Yu Kang, Dan Li, Peichen Pan, Yafeng Deng, Dongsheng Cao, Chang-Yu Hsieh, and Tingjun Hou. 2023. Chemistry-intuitive explanation of graph neural networks for molecular property prediction with substructure masking. *Nature Communications* 14, 2585 (2023).

[58] Yikun Xian, Zuohui Fu, S. Muthukrishnan, Gerard de Melo, and Yongfeng Zhang. 2019. Reinforcement Knowledge Graph Reasoning for Explainable Recommendation. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*.

Appendix

Explanation for Fractional z_w. Since x_u remains binary in the MILP, the linking constraints force each d_e to be binary as well. Specifically, if an edge e is contained in a selected intervention unit, then $d_e \geq x_u = 1$, and hence $d_e = 1$. Otherwise, $\sum_{u:e\in E_u} x_u = 0$, so $d_e = 0$.

It remains to consider z_w. Since $\sum_{w\in W} z_w = 1$ and $z_w \geq 0$, there exists at least one match $r \in W$ with $z_r > 0$. For every edge $e \in E_r$, the constraint $z_r + d_e \leq 1$ implies $d_e < 1$. Because d_e is already forced to be binary, we obtain $d_e = 0$. Hence no edge of r is deleted, and r is a surviving query match.

Therefore, any match with positive z_w is a valid survivor. We may choose one such match r, set $z_r = 1$, and set all other $z_w = 0$ without changing x, d, or the objective value. Thus, relaxing z_w to $0 \leq z_w \leq 1$ does not affect the exactness of the MILP, provided that x_u remains binary.

Procedures for Algorithms. The two procedures Lookup and SelectUnit used for both LpRdRes and LpRdRsp are shown below.

Algorithm 3 Lookup(W, U)

Require: Query matches W and intervention units U
Ensure: Sparse edge-to-unit lookup $\mathcal{L}$

```
1:  E_W ← ∅
2:  for all w ∈ W do
3:      E_W ← E_W ∪ E_w
4:  for all e ∈ E_W do
5:      L(e) ← ∅
6:  for all u ∈ U do
7:      for all e ∈ E_u do
8:          if e ∈ E_W then
9:              L(e) ← L(e) ∪ {u}
10: return L
```

The procedure Lookup constructs a sparse edge-to-unit index for efficiently identifying intervention units that overlap a query match. It first collects the set $E_W = \bigcup_{w\in W} E_w$ of edges occurring in at least one query match (lines 1–3). For each edge $e \in E_W$, it initializes an empty lookup entry $\mathcal{L}(e)$ (lines 4–5). It then scans every intervention unit $u \in U$ and inserts u into $\mathcal{L}(e)$ for each edge $e \in E_u \cap E_W$ (lines 6–9). Consequently, the resulting lookup satisfies $\mathcal{L}(e) = \{u \in U \mid e \in E_u\}$ for every edge e occurring in a query match. The lookup is returned (line 10). The lookup avoids explicitly materializing the incidence relation between all intervention units and all query matches, which may require $O(|U||W|)$ space. Instead, $\mathcal{L}$ stores only actual edge-to-unit incidences. Assuming constant-time set operations, its construction requires $O\left(\sum_{w\in W} |E_w| + \sum_{u\in U} |E_u|\right)$ time and $O\left(|E_W| + \sum_{e\in E_W} |\mathcal{L}(e)|\right)$ space.

Algorithm 4 SelectUnit$(w, \mathcal{L})$

Require: An undestroyed query match w and lookup $\mathcal{L}$.
Ensure: An intervention unit overlapping w.

```
1: for all e ∈ E_w do
2:     if L(e) ≠ ∅ then
3:         return any u ∈ L(e)
```

Given an undestroyed query match w, the procedure SelectUnit scans its edges (line 1). For an edge $e \in E_w$, every intervention unit $u \in \mathcal{L}(e)$ contains e. Hence, $e \in E_u \cap E_w$, and deleting u destroys w. Therefore, once a nonempty lookup entry $\mathcal{L}(e)$ is encountered, the procedure returns an arbitrary unit from that entry (line 3).

All units stored in $\mathcal{L}$ are valid candidates because the lookup is constructed from the permitted intervention-unit set U. For responsibility, the same procedure can be used after constructing the lookup from the restricted candidate set.